
Algorithm 1 Meta-evolution of symbolic-regression operators

Input Baseline bundle B_0 ; training tasks $\mathcal{T}$; fitness F ; population target M ; offspring count O ; G generations, including G_s final simplification generations; initial and survivor seed counts $n_0 \leq n_1$.

```
1  $\mathcal{P} \leftarrow \{B_0\} \cup \text{INITIALEXPLORE}(B_0, M - 1)$ 
2  $\mathcal{A} \leftarrow \text{EVALUATE}(\mathcal{P}, \mathcal{T}, F, n_0)$  # Store per-task, per-seed scores
3 for  $g = 1, \dots, G$  do
4    $s \leftarrow [g > G - G_s]$  # Switch from performance to simplification
5   if  $s$  and this is the first simplification generation then
6      $\mathcal{P} \leftarrow \text{SEEDFRONTIER}(\mathcal{A}, M)$  # Recover the archive's score-LOC tradeoffs
7   Launch survivor top-ups to  $n_1$  seeds;  $\mathcal{C} \leftarrow \emptyset$ 
8   for  $j$  in a shuffled schedule of  $O$  slots, balanced across operator types do
9      $B \leftarrow \text{TOURNAMENT}(\mathcal{P}, 2)$  # Higher mean fitness wins
10     $m \leftarrow \text{simplify}$  if  $s$ , otherwise a uniform feasible meta-mutation mode
11     $U \leftarrow \text{SOURCEOPERATORS}(\mathcal{P}, B, j, m)$ 
12     $q \leftarrow \text{PROMPT}(j, m, U, \text{OPTIONALTRACEFEEDBACK}(B, \mathcal{T}))$ 
13     $u \leftarrow \text{VALIDATEDLLMCOMPLETION}(q)$  # Sample ensemble; retry invalid code
14     $\mathcal{C} \leftarrow \mathcal{C} \cup \{B[j \leftarrow u]\}$  # Replace one slot; inherit the rest
15  Evaluate  $\mathcal{C}$  on  $n_0$  seeds, overlapping survivor top-ups
16  Merge new observations into  $\mathcal{A}$ ; update task means and overall fitness
17   $\mathcal{P} \leftarrow \begin{cases} \text{SCORELOCSURVIVORS}(\mathcal{P} \cup \mathcal{C}, M), & s, \\ \text{TASKDIVERSESURVIVORS}(\mathcal{P} \cup \mathcal{C}, M), & \text{otherwise} \end{cases}$ 
18  Top up the last survivors to  $n_1$  seeds
19   $\mathcal{Q} \leftarrow \text{FINALISTS}(\mathcal{A}, \mathcal{P})$ ; evaluate  $\mathcal{Q}$  on fresh shared seeds
20 return fresh scores and  $\arg \max_{B \in \mathcal{Q}} \hat{F}_{\text{fresh}}(B)$ 
```

Modes. Explore, refine, crossover, simplify. Refine/simplify use the selected bundle's operator when available; crossover samples two source operators uniformly. Optional trace feedback concerns unsolved tasks. Failed or infeasible proposals use the implementation's bounded retries and fallbacks.

Survivors. Task diversity retains distinct best positive-rate solvers, then backfills to at least M by overall fitness. Score-LOC selection takes a best candidate per LOC bucket, Pareto-filters these winners, then backfills to M by fitness. The archive mean is $\hat{F}(B) = |\mathcal{T}|^{-1} \sum_{t \in \mathcal{T}} n_B^{-1} \sum_{r=1}^{n_B} F(B, t, r)$.

Scope. The task-diverse, population-reevaluation variant is shown. Finalists may be the terminal population or an archive shortlist; the current identification pass uses the top K code-distinct archive bundles by shrinkage-adjusted training score. Held-out validation and benchmark evaluation are separate.

Algorithm 2 PySR search with four evolvable operator slots

Input Data $D = (X, y)$; expression grammar; search budget; populations $P_1, \dots, P_J$; size limit $c_{\max}$; tournament size k ; crossover probability $p_{\times}$.

```
1 Initialize and score random expressions; initialize hall of fame  $H$  and size frequencies  $f$ 
2 while search budget and iteration allowance remain do
3   Choose a population  $P$  to evolve # Population workers may run concurrently
4   for each inner cycle, with scheduled temperature  $T$  do
5     for  $e = 1, \dots, \lceil |P|/k \rceil$  do
6       if  $\text{Unif}(0, 1) < p_{\times}$  then
7          $a, b \leftarrow \text{Select}(P, f), \text{Select}(P, f)$ 
8          $C \leftarrow \text{CONSTRAINEDSUBTREECROSSOVER}(a, b)$ 
9         Score both children using Loss $(\cdot, D)$  and the cost transform
10        else
11           $a \leftarrow \text{Select}(P, f)$ 
12          Draw a feasible mutation from the weighted menu, including Mutate
13           $C \leftarrow \text{PROPOSEANDACCEPT}(a, T, f, c_{\max}, \text{Loss})$ 
14          if the proposal succeeds then
15             $I \leftarrow \text{DISTINCTVICTIMS}(P, |C|, \text{Survive})$ 
16            Replace members at  $I$  with  $C$ , assigning offspring birth times and lineage
17          Retain the best expression at each complexity in a local hall of fame
18          Simplify expressions; probabilistically optimize constants; refresh scores
19          Merge local and population champions into  $H$ ; update size frequencies  $f$ 
20          If enabled, inject migrants from population elites and the Pareto frontier of  $H$ 
21          Advance the size-limit schedule and the frequency window
22 return the loss–complexity Pareto frontier of  $H$ 
```

Bold sans-serif marks an evolvable operator slot. When supplied, the mutation slot adds a move to PySR’s weighted menu (tree edits, constant changes, simplification, and other configured moves); selection, survival, and loss replace their respective hooks. **Survive** returns a replacement index, not a new population. For crossover, both victims are chosen before insertion, with the first index excluded from the second call.

Baseline hooks. Loss is the configured pointwise regression loss (MSE by default); cost normalizes loss and adds parsimony. Selection ranks a size- k sample by cost, optionally multiplied by $\exp(\lambda f_c)$, and draws rank r with probability proportional to $p(1 - p)^{r-1}$. Survival removes the oldest eligible member.

Acceptance. Ordinary tree mutations pass constraints and a non-NaN cost check, then optional annealing and size-frequency tests. Their acceptance probability is $\min\{1, \exp[-(C_{\text{child}} - C_{\text{parent}})/(\alpha T)] f_{c_{\text{parent}}}/f_{c_{\text{child}}}\}$ when both options are enabled. Crossover uses constraints without this test. Some built-in moves return immediately. The default failure policy skips unsuccessful proposals.

Algorithm 3 BasicSR / FullSR seed: a search engine with eight policy slots

Input Data D ; grammar; search budget; J populations of size N ; K inner cycles per population pass.

```
1 Initialize policy state  $S$  and random populations  $\mathcal{P} = (P_1, \dots, P_J)$ 
2 Score initial trees with Loss; call UpdateState( $\mathcal{P}, S$ )
3 while search budget and iteration allowance remain do
4   for each population  $P_j$  do
5     for  $c = 1, \dots, K$  do
6       UpdateState( $\mathcal{P}, S$ )
7       for  $e = 1, \dots, \lceil |P_j|/15 \rceil$  do
8         if  $\text{Unif}(0, 1) \leq 0.0259$  and  $|P_j| \geq 2$  then
9            $a, b \leftarrow \text{Select}(P_j, S), \text{Select}(P_j, S)$ 
10          If  $a = b$ , use the next population member as  $b$ 
11           $T \leftarrow \text{Crossover}(a, b, S)$  # Up to two trees, with their parents
12        else
13           $a \leftarrow \text{Select}(P_j, S); T \leftarrow \text{Mutate}(a, S)$ 
14           $C \leftarrow \emptyset$ 
15          for each proposed tree  $t$  and corresponding parent  $a$  do
16             $u \leftarrow \text{MAKEINDIVIDUAL}(t, \text{Loss}(t, D, S))$ 
17            If Accept( $a, u, S$ ), add  $u$  to  $C$ 
18          If  $C \neq \emptyset$ , set  $P_j \leftarrow \text{Survive}(P_j, C, S)$ 
19          Run configured simplification and constant optimization on  $P_j$ 
20          Mark the population pass complete; UpdateState( $\mathcal{P}, S$ )
21           $\mathcal{P} \leftarrow \text{UpdatePopulation}(\mathcal{P}, S)$ 
22 return the policy archive, ordered by complexity (or the best member if empty)
```

Slot	Baseline policy
Loss	MSE; cost equals loss. Invalid or extreme predictions give $+\infty$.
Select	Best of $\min(15, P)$ distinct uniformly sampled members.
Mutate	Replace a uniform node by a terminal (probability 1/2) or a shallow random operator.
Crossover	Swap uniformly chosen subtrees; up to 10 constraint-validity attempts.
Accept	Accept every constructed child.
Survive	Keep the best N from parents and children, ranked by cost.
UpdateState	Keep 10 distinct finite-loss expressions; refresh after completed passes.
UpdatePopulation	Identity: no migration.

Implementation distinction. BasicSR enables simplification and constant optimization (probability 0.14). The current FullSR seed in `SRConfig.jl` lacks those state options, so the same engine's postprocessing hook is inactive. The eight baseline policy functions otherwise implement the same search choices. Mutation also allows up to 10 validity attempts; failed proposals produce no child.

Algorithm 4 Evolved mutation: duplicate and couple a reusable motif

Input Expression tree t ; available binary operators $\mathcal{O}_2$; maximum node count L ; number of features d .

```
1  $\mathcal{O}_a \leftarrow \{+, \times\} \cap \mathcal{O}_2$ 
2  $\delta \leftarrow -$  if available, otherwise  $+$  if available, otherwise  $\emptyset$ 
3  $r_{\text{ok}} \leftarrow [/\in \mathcal{O}_2] \wedge [\delta \neq \emptyset] \wedge [L - |t| - 3 \geq 1]$ 
4  $r \leftarrow r_{\text{ok}} \wedge (\mathcal{O}_a = \emptyset \vee \text{Unif}(0, 1) < 0.35)$ 
5 If  $\neg r$  and  $\mathcal{O}_a = \emptyset$ , return  $t$ 
6  $b \leftarrow L - |t| - (3 \text{ if } r \text{ else } 1)$  # Node budget for the donor
7  $\mathcal{D} \leftarrow \{u \text{ subtree of } t : |u| \leq b, u \text{ contains a variable}\}$ 
8 If  $\mathcal{D} = \emptyset$ , return  $t$ 
9  $\mathcal{D}_+ \leftarrow \{u \in \mathcal{D} : u \text{ is not a leaf}\}$ 
10  $v \leftarrow \text{COPY}(\text{Unif}(\mathcal{D}_+ \text{ if nonempty, else } \mathcal{D}))$ 
11  $u \leftarrow \text{a uniformly sampled target node of } t$ 
12 if  $d > 1$  and  $\text{Unif}(0, 1) < 1/2$  then
13 |  $h \leftarrow \text{Unif}\{1, \dots, d-1\}$ 
14 | In  $v$ , replace every feature  $x_j$  by  $x_{1+((j-1+h) \bmod d)}$ 
15 if  $r$  then
16 | Replace  $u$  by  $\text{COPY}(u)/(1 \delta v)$  # Rational correction
17 else
18 |  $\_ \circ \leftarrow \text{Unif}(\mathcal{O}_a)$ ; replace  $u$  by  $\text{COPY}(u) \circ v$ 
19 return  $t$ 
```

The donor is copied before editing the target, so the two may overlap in the original tree. Compound, variable-bearing donors are preferred; the cyclic shift reuses their structure across features. The 0.35 rational probability applies when rational coupling is feasible and an accumulator exists. If rational coupling is the only option, it is always chosen. The outer PySR mutation machinery still checks all constraints.

Algorithm 5 Evolved survival: replace an old, costly member

Input Population P with birth indices b_i and costs c_i ; excluded replacement indices E . At least one index is eligible.

```
1  $I \leftarrow \{1, \dots, |P|\} \setminus E$ 
2  $b_- \leftarrow \min_{i \in I} b_i$ ;  $b_+ \leftarrow \max_{i \in I} b_i$ 
3  $c_- \leftarrow \min_{i \in I} c_i$ ;  $c_+ \leftarrow \max_{i \in I} c_i$ 
4 for  $i \in I$  do
5    $a_i \leftarrow \begin{cases} (b_+ - b_i)/(b_+ - b_-), & b_+ \neq b_-, \\ 1, & \text{otherwise} \end{cases}$ 
6    $q_i \leftarrow \begin{cases} (c_i - c_-)/(c_+ - c_-), & c_+ \neq c_-, \\ 0, & \text{otherwise} \end{cases}$ 
7    $v_i \leftarrow 0.75 a_i + 0.25 q_i$ 
8 return the first index in  $\arg \max_{i \in I} v_i$ 
```

Smaller birth indices denote older members. Both age and cost are normalized over the eligible population: old age and high cost increase replacement priority. This is a weighted tradeoff, rather than a strict age ordering with a cost tie-break. The returned index is replaced by PySR; the operator does not insert the child.

The equations describe the source's raw min-max arithmetic. It has no special handling for nonfinite costs.

Algorithm 6 Evolved selection: suppress clones and explore quality-gated niches

Input Population P ; tournament size k ; geometric parameter p ; maximum complexity L ; global normalized complexity frequencies f_c ; parsimony scale λ .

```
1  $S \leftarrow$  uniformly sample  $\min(k, |P|)$  members without replacement
2 for  $i \in S$  do
3    $c_i \leftarrow \text{COMPLEXITY}(i)$ ;  $\ell_i \leftarrow \text{LOSS}(i)$ 
4    $s_i \leftarrow \text{COST}(i) \exp(\lambda f_{c_i})$  if frequency weighting is enabled,
   otherwise  $s_i \leftarrow \text{COST}(i)$ ; use  $f_c = 0$  outside  $1 \leq c \leq L$ 
5  $\pi \leftarrow \text{ARGSORT}(s)$ ;  $\tilde{s} \leftarrow s$ 
6 for  $a = 2, \dots, |S|$  do
7    $i \leftarrow \pi_a$ 
8   if some  $j \in \{\pi_1, \dots, \pi_{a-1}\}$  satisfies  $c_i = c_j$  and  $|\ell_i - \ell_j| \leq 10^{-6} \max(|\ell_i|, |\ell_j|, \epsilon)$  then
9      $\tilde{s}_i \leftarrow \begin{cases} 1.35s_i, & s_i > 0, \\ s_i/1.35, & s_i < 0, \\ 0, & s_i = 0 \end{cases}$  # One sign-safe penalty per clone
10   $\rho \leftarrow \text{ARGSORT}(\tilde{s})$ 
11  if  $\text{Unif}(0, 1) < 0.10$  then
12     $H \leftarrow$  first member of each valid complexity in  $\rho$  order
13    if  $H \neq \emptyset$  then
14       $H_q \leftarrow$  first  $\lceil |H|/2 \rceil$  members of  $H$  # Keep the better half
15      return  $\arg \min_{i \in H_q} (f_{c_i}, \tilde{s}_i)$  # Rarest niche; quality breaks ties
16  Draw rank  $r \in \{1, \dots, |S|\}$  with  $\Pr(r) \propto p(1-p)^{r-1}$ 
17 return  $S_{\rho_r}$  # For  $p = 1$ , choose the best adjusted score
```

Clone detection uses complexity and nearly equal loss as a phenotype proxy; it does not compare expression strings. All earlier members in the *original* ranking can establish a clone match, including earlier clones. Niche rarity uses global search frequencies, even when frequency weighting of tournament costs is disabled. If no valid niche exists, selection falls back to the geometric tournament.

Algorithm 7 Evolved loss: fit functional shape, then reward calibration

Input Expression t and data (X, y) with n examples.

```
1  $z \leftarrow t(X)$ ; return  $+\infty$  if evaluation fails, lengths disagree, or  $n = 0$ 
2 Accumulate  $\sum_i z_i, \sum_i y_i, \sum_i z_i^2, \sum_i y_i^2, \sum_i z_i y_i$ 
3  $\bar{z} \leftarrow n^{-1} \sum_i z_i; \bar{y} \leftarrow n^{-1} \sum_i y_i$ 
4  $V_z \leftarrow \max(\sum_i z_i^2 - n\bar{z}^2, 0); V_y \leftarrow \max(\sum_i y_i^2 - n\bar{y}^2, 0)$ 
5  $C_{zy} \leftarrow \sum_i z_i y_i - n\bar{z}\bar{y}$ 
6 Return  $+\infty$  if any centered moment is nonfinite
7  $\sigma \leftarrow \sqrt{V_y/n}$  if  $V_y > 0$ , otherwise  $\max(1, |\bar{y}|)$ 
8  $a \leftarrow C_{zy}/V_z$  if  $V_z > 0$ , otherwise 0;  $b \leftarrow \bar{y} - a\bar{z}$ 
9 Return  $+\infty$  if  $a$  or  $b$  is nonfinite
10  $A \leftarrow 0; R \leftarrow 0$  # Second pass: directly evaluate residuals
11 for  $i = 1, \dots, n$  do
12    $A \leftarrow A + ((az_i + b - y_i)/\sigma)^2$ 
13    $R \leftarrow R + ((z_i - y_i)/\sigma)^2$ 
14  $q \leftarrow R/n; h \leftarrow q/(1 + q)$  if  $q$  is finite, otherwise 1
15  $\ell \leftarrow \sqrt{\text{clip}(A/n, 0, 1)} + h/256$ 
16 return  $\max(\ell, 0)$  if finite, otherwise  $+\infty$ 
```

For a nonconstant target, the principal term is the square root of the normalized MSE after the best affine fit $az + b$. The bounded raw-error term favors already calibrated expressions and contributes at most $1/256$. The fitted coefficients are used only to score the tree: this operator does not rewrite it as $at + b$.

The first pass uses plain power sums, with centered variances floored at zero. The second pass evaluates fitted residuals directly (the Julia source uses fused multiply-add) to reduce cancellation near a shape match. Constant predictions use a mean-only fit; constant targets use the magnitude scale $\max(1, |\bar{y}|)$.
